Informe — Mesa de Pedidos (Desafío 1)

1. Cómo se desarrolló

La decisión

El sistema se construyó con un agente de código (Claude Code). El código lo escribió el agente; las decisiones, el diseño y la validación fueron míos. Lo elegí así para invertir el tiempo en lo que el desafío evalúa —entender RAG, tools, grafo, estado e `interrupt`, y cómo se arma un sistema con agentes— en lugar de aprender Python a fondo, que hubiera llevado mucho más tiempo sin sumar a esos conceptos.

El ciclo de cada pieza

El agente no implementaba nada sin que yo lo hubiera razonado antes. Para cada pieza implementada seguimos estos pasos:

- **Razonar:** explicaba con mis palabras qué recibe, qué decide, qué devuelve y qué pasa cuando falla. El agente evaluaba ese razonamiento contra la consigna, la política y casos reales de la bandeja, y cuando había un hueco me devolvía una repregunta en lugar de la respuesta.
- **Acordar el contrato:** una tabla corta con entradas, salidas, campos del estado y forma de fallar, que yo confirmaba o modificaba antes de escribir código.
- **Implementar y leer:** el agente escribía el código mínimo y después lo recorríamos bloque por bloque, mapeando cada paso de mi explicación a las líneas que lo cumplen.
- **Predecir y ejecutar:** antes de correr, decía qué esperaba ver (qué rama, qué campos, qué cita). La diferencia entre predicción y resultado era lo que me daba una idea de lo que me faltaba entender.
- **Cerrar con preguntas** de concepto sin mirar el código para entender lo implementado.

2. Cómo correrlo

Ver el README: instalación, configuración (embeddings locales o por API) y cómo probar el grafo.

3. Decisiones y por qué

A medida que avanzaba en la implementación fui anotando cada decisión en BITACORA.md: las definiciones tomadas de la política, cada decisión técnica con su porqué y las alternativas descartadas, los conceptos aplicados y lo que quedó fuera de alcance. Acá resumo las principales; el detalle está en la bitácora.

Dónde va el LLM

Usé el LLM sólo donde hay texto libre que interpretar: para clasificar el mensaje, para extraer el pedido y para redactar la respuesta a una consulta. La clasificación y la extracción devuelven structured output con Pydantic, así que una etiqueta fuera de la lista se rechaza en el momento en lugar de romper el router más adelante. Si la salida no se puede leer, el mensaje se deriva: ante la duda, prefiero el lado seguro.

`validar_pedido`, en cambio, no usa LLM. Precio, stock y crédito salen de los CSV a través de las tools, y las reglas son condiciones simples con los valores de la política en constantes. Con un LLM en ese nodo, el mismo pedido podría recibir veredictos distintos.

Chunking por sección

Empecé cortando los PDF por tamaño fijo (1000 caracteres, 150 de solapamiento) y la tabla de zonas de flete quedaba partida: MSG-001 fallaba porque no se recuperaba el fragmento correcto. Con chunks de 1500 ese caso se arreglaba, pero la regla de vigencia del presupuesto se diluía en un fragmento más largo y fallaba MSG-005. Ahí entendí que el problema no era el tamaño sino cortar en medio de una sección. Ahora corto primero por sección del documento y sólo subdivido las que superan los 1000 caracteres. Cada fragmento conserva documento y página para poder citarlos.

Lo que la política no define

En dos puntos tuve que interpretar. Los umbrales se comparan contra el monto neto: precio de lista por cantidad, sin IVA y antes de descuentos, porque el art. 4 elige el descuento por volumen a partir de ese monto. El descuento total es la suma del habitual, el de volumen y el que pide el cliente, y se compara contra el tope (20%, o 15% si hay herramienta eléctrica de marca). Ese control sólo manda el pedido a Gerencia si el cliente pidió un descuento; si no lo pidió, la Mesa aplica hasta el tope sin consultar (MSG-010).

Qué llevó más tiempo

A la hora de realizar la implementación, uno de los ítems que más tiempo me llevó entender y ajustar fue el chunking del RAG. Al ajustarlo partía un documento por lo que había respuestas incorrectas o que no alcanzaba el contexto para responderlas. Al agrandar el chunk arreglaba una pero otra regla quedaba diluida en un chunk más largo y la búsqueda traía un fragmento que no tenía el dato. Finalmente me di cuenta que el problema no era el tamaño de chunks sino respetar la estructura del documento separándolo por secciones con los títulos que ya trae: los artículos ("ART. N -").

También me llevaron tiempo las reglas de `validar_pedido`, porque la política no define todo: hubo que interpretar el monto neto, cómo se suma el descuento, qué casos se rechazan "sin excepción" y cuáles van a Gerencia, y qué salida gana cuando se disparan varias reglas a la vez.

4. Fuera de alcance

Qué	Por qué
Acumular pedidos bajo el mínimo durante 30 días (art. 3)	Exige memoria entre mensajes, escribir datos y un disparador por tiempo
Ejecutar entregas parciales	Las decide Gerencia; el sistema no arma el pedido parcial
Un pedido y una consulta aparte en el mismo mensaje	No lo contempla la consigna
Productos pedidos sin SKU	Ningún pedido de la bandeja lo trae
Sugerir SKU parecidos ante uno inexistente	Sólo sirve si un agente elige las tools

Mejoras futuras

- Que Gerencia pueda dejar un comentario libre, y ver el crédito actual junto al del momento en que se frenó el pedido.
- Avisar al indexar si un PDF no tiene títulos detectados, porque entonces se corta por tamaño.